

A Method for Computing Inverse Parametric PDE Problems with Random-Weight Neural Networks

Summary of Dong and Wang (2023)

1 Introduction

This paper presents a method for solving inverse parametric partial differential equations (PDEs) using randomized neural networks, extending the local extreme learning machine (locELM) technique to inverse problems. The method focuses on simultaneously determining unknown parameters (as constants or field distributions) and the solution field to parametric PDEs based on sparse and potentially noisy measurement data.

2 Algorithm Description

2.1 Problem Formulation

The method addresses inverse problems described by parametric PDEs of the form:

$$\alpha_1 L_1(u) + \alpha_2 L_2(u) + \cdots + \alpha_n L_n(u) + F(u) = f(x), \quad x \in \Omega \quad (1)$$

$$Bu(x) = g(x), \quad x \in \partial\Omega \quad (2)$$

$$Mu(\xi) = S(\xi), \quad \xi \in \Omega_s \subset \Omega \quad (3)$$

where:

- L_i and F are differential or algebraic operators (linear or nonlinear)
- $u(x)$ is the unknown scalar field to be determined
- α_i are n unknown parameters (constants or field distributions)
- B is a linear operator representing boundary conditions
- M is an operator representing measurement operations
- $S(\xi)$ is the measurement data at points $\xi \in \Omega_s$

2.2 Local Randomized Neural Networks

The domain Ω is decomposed into non-overlapping sub-domains $\Omega = \Omega_1 \cup \Omega_2 \cup \dots \cup \Omega_N$, and the solution field $u(x)$ is represented by:

$$u(x) = \begin{cases} u_1(x), & x \in \Omega_1 \\ u_2(x), & x \in \Omega_2 \\ \vdots \\ u_N(x), & x \in \Omega_N \end{cases} \quad (4)$$

On the interior sub-domain boundaries, C^k continuity conditions are imposed on $u(x)$, where k relates to the order of the PDE.

On each sub-domain Ω_i , a local feedforward neural network (FNN) is employed with the following characteristics:

- Input layer: d nodes representing the input coordinates $x = (x_1, x_2, \dots, x_d)$
- Output layer: Single node representing $u_i(x)$
- Hidden layers: $(L - 1)$ hidden layers in between
- Architectural vector: $M = [m_0, m_1, \dots, m_L]$ where $m_0 = d$ and $m_L = 1$
- The weights/biases in all hidden layers are randomly assigned on $[-R_m, R_m]$ and fixed (not trainable)
- Only the output-layer weights are trainable parameters

For each sub-domain Ω_i , the solution is represented as:

$$u_i(x) = \sum_{j=1}^M \beta_{ij} \phi_{ij}(x) = \Phi_i(x) \beta_i \quad (5)$$

where $M = m_{L-1}$ is the width of the last hidden layer, $\phi_{ij}(x)$ are the output fields of the last hidden layer, and β_{ij} are the trainable output-layer coefficients.

2.3 Three Training Algorithms

The paper develops three algorithms for training the neural network:

2.3.1 NLLSQ Algorithm

The Nonlinear Least Squares (NLLSQ) algorithm determines the inverse parameters α and the trainable parameters β simultaneously using a nonlinear least squares method with perturbations (NLLSQ-perturb).

The residual vector is defined as:

$$R(\alpha, \beta) = \begin{bmatrix} R_{pde}(\alpha, \beta) \\ R_{bc}(\beta) \\ R_{mea}(\beta) \\ R_{ck}(\beta) \end{bmatrix} \quad (6)$$

This system is solved using nonlinear least squares combined with a perturbation strategy to avoid being trapped in local minima. The key features of this algorithm include:

- Initial guess perturbation: If the returned solution exceeds a tolerance, a sub-iteration procedure is triggered with new initial guesses
- Forward-mode auto-differentiation is used for computing derivatives, which is crucial for performance
- Measurement residual scaling: For noisy data, scaling the measurement residual vector by a factor λ_{mea} can improve accuracy

2.3.2 VarPro-F1 Algorithm

The Variable Projection Formulation 1 (VarPro-F1) algorithm eliminates the inverse parameters α from the problem to attain a reduced problem about β only:

- The system is rearranged into $H(\beta)\alpha = b(\beta)$
- For any given β , the least squares solution for α is $\alpha = H^+(\beta)b(\beta)$
- Substituting this expression gives a reduced residual $r(\beta) = H(\beta)H^+(\beta)b(\beta) - b(\beta)$
- The optimal β^* is determined by minimizing $\frac{1}{2}\|r(\beta)\|^2$
- Once β is determined, α is computed by linear least squares

2.3.3 VarPro-F2 Algorithm

The Variable Projection Formulation 2 (VarPro-F2) algorithm eliminates the trainable network parameters β from the problem to attain a reduced problem about α only:

- The system is rearranged into $H(\alpha)\beta = b$

- For any given α , the least squares solution for β is $\beta = H^+(\alpha)b$
- Substituting this expression gives a reduced residual $r(\alpha) = H(\alpha)H^+(\alpha)b - b$
- The optimal α^* is determined by minimizing $\frac{1}{2}\|r(\alpha)\|^2$
- Once α is determined, β is computed by linear least squares

For PDEs that are nonlinear with respect to the field solution, VarPro-F2 needs to be combined with a Newton iteration.

3 Theoretical Findings

3.1 Exponential Convergence Properties

A significant theoretical finding is that for smooth solutions and noise-free data, the errors for the inverse parameters and the solution field decrease exponentially with respect to:

- The number of collocation points
- The number of training parameters

This exponential convergence is analytically demonstrated and empirically verified across multiple test cases. For sufficiently large parameter values, the errors can reach a level close to machine accuracy.

3.2 Random-Weight Neural Networks as Universal Approximators

The method leverages the proven theoretical results that single-hidden-layer feedforward neural networks with random but fixed hidden units can approximate any continuous function to any desired degree of accuracy, provided that the number of hidden units is sufficiently large.

3.3 Properties of the Variable Projection Formulation

The theoretical benefits of the variable projection approach include:

- Reduced dimension of parameter space
- Better conditioning of the optimization problem
- Faster convergence with the reduced problem

The VarPro-F1 and VarPro-F2 algorithms are reciprocal formulations of the variable projection idea, with the former being more suitable for general cases and the latter being more efficient for linear (with respect to solution field) parametric PDEs.

4 Numerical Examples and Results

The method was rigorously tested using several inverse parametric PDE problems:

4.1 Parametric Poisson Equation

The inverse problem of determining the coefficient α in:

$$\frac{\partial^2 u}{\partial x^2} + \alpha \frac{\partial^2 u}{\partial y^2} = f(x, y) \quad (7)$$

$$\text{with boundary conditions and measurement data} \quad (8)$$

Key findings:

- For noise-free data, the maximum error for u was on the order of 10^{-7} in the domain
- The relative error of the computed α was approximately 9.03×10^{-9}
- Exponential convergence with respect to the number of collocation points
- Exponential convergence with respect to the number of training parameters
- With 10% noise in the measurement data, the relative error of α was around 1%

4.2 Parametric Nonlinear Helmholtz Equation

The inverse problem of determining coefficients α_1 and α_2 in:

$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} - \alpha_1 u + \alpha_2 \cos(2u) = f(x, y) \quad (9)$$

$$\text{with boundary conditions and measurement data} \quad (10)$$

Key findings:

- VarPro-F1 exhibited high accuracy with maximum error on the order of 10^{-8}
- The relative errors of α_1 and α_2 were 3.52×10^{-11} and 4.76×10^{-10}
- Exponential convergence with respect to collocation points and training parameters
- With 1% noise, relative errors for α_1 and α_2 were on the order of 0.1%

4.3 Parametric Viscous Burgers' Equation

The inverse problem of determining α_1 and α_2 in:

$$\frac{\partial u}{\partial t} + \alpha_1 u \frac{\partial u}{\partial x} = \alpha_2 \frac{\partial^2 u}{\partial x^2} + f(x, t) \quad (11)$$

with boundary/initial conditions and measurement data (12)

Key findings:

- The maximum error for u was on the order of 10^{-8} in the domain
- The relative errors of α_1 and α_2 were 1.30×10^{-9} and 1.48×10^{-8}
- Exponential convergence with respect to collocation points and training parameters
- With 1% noise, relative errors were around 0.026% for α_1 and 0.2% for α_2

4.4 Helmholtz Equation with Inverse Variable Coefficient

An inverse problem involving an unknown coefficient field:

$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} - \gamma(x, y)u = f(x, y) \quad (13)$$

with boundary conditions and measurement data (14)

Key findings:

- High accuracy with maximum u error on the order of 10^{-8} and maximum γ error on 10^{-5}
- Exponential decrease in errors with respect to collocation points, training parameters, and measurement points
- For 1% noise, the maximum relative error for $\gamma(x, y)$ was around 43% (l_∞ norm) and 5% (l_2 norm)
- Regularization of output-layer coefficients improved accuracy in the presence of noise

5 Implications and Advantages

5.1 Computational Performance

- Network training time grows approximately linearly with the number of collocation points and training parameters

- The method is significantly faster and more accurate than Physics-Informed Neural Networks (PINN) for inverse problems
- For noise-free data, the method achieves accuracy several orders of magnitude higher than PINN

5.2 Robustness to Noise

- The method remains accurate even with noisy measurement data
- Scaling the measurement residual by a factor λ_{mea} ($0 \leq \lambda_{mea} \leq 1$) markedly improves accuracy for noisy data
- The robustness varies depending on the complexity of the inverse problem

5.3 Practical Applications

The method has significant implications for real-world applications where:

- The physics is partially known (governing equations with unknown parameters)
- Only sparse measurements are available
- Parameter field distributions need to be identified

5.4 Algorithm Selection

The paper provides guidance on algorithm selection:

- NLLSQ: Generally applicable to all problems, balance of efficiency and accuracy
- VarPro-F1: Effective for general cases, slightly more costly but can be more robust
- VarPro-F2: Most efficient for linear (with respect to solution) PDEs, but requires Newton iteration for nonlinear PDEs

The exponential convergence exhibited by the method suggests it could be particularly valuable for high-accuracy requirements in scientific computing and engineering applications.